1. Práctica 14: Creación automática de BSP

1.1. Objetivos

- Aprender a utilizar herramientas avanzadas de desarrollo de sistemas empotrados como STMCubeMX.
- Comprender la planificación de pines al utilizar un microcontrolador.

1.2. Fundamentos teóricos

Hasta ahora, hemos estado desarrollando nuestros propios paquetes BSP para trabajar con nuestra placa. Así, hemos conseguido facilitar el uso mediante funciones auxiliares que nos permitan realizar configuraciones o acciones. Para saber exactamente cómo trabajar con la placa, hemos tenido que consultar `rm0431`, `um2140`, `ds11853`, `pm0253`, `an2606`, ...

Normalmente (y así es en el caso de ST, desarrolladora del **STM32F723IE**), se proporcionan paquetes software que ya contienen toda la funcionalidad preprogramada, para evitarnos tener que hacerlo manualmente. Estos paquetes son desarrollados por los ingenieros de "software", basándose en los manuales proporcionados por los ingenieros de "hardware".

En ocasiones, se proporciona un único paquete genérico con toda la funcionalidad, y es responsabilidad del usuario final decidir qué y cómo lo utiliza. Aunque esto tiene la ventaja de empaquetar toda la funcionalidad en un único conjunto de librerías, tiene el inconveniente de agrandar el código al incluir mucha funcionalidad que quizá no utilicemos. En el caso de ST, proporcionan una herramienta para crear un BSP a medida con únicamente los elementos que queramos incluir. Es esta funcionalidad la que vemos a continuación.

1.3. Desarrollo de la práctica

1.3.1. Creación del BSP con STMCubeMX

Lo primero de todo es abrir la aplicación STMCubeMX (Figura 1). Tenemos varias funciones:

- Ver los proyectos ya creados.
- Empezar un proyecto eligiendo un procesador, una placa, o un ejemplo preparado por ST.
- Actualizar la aplicación e incluir paquetes software nuevos.

Lo primero que haremos, si es que no está ya instalado, es descargar el paquete STM32F7 desde el apartado "Install or remove embedded software packages" (Figura 2).

A continuación, debemos elegir el componente base de nuestro proyecto. Tenemos dos opciones:

- **MCU**: Empieza el proyecto partiendo de un procesador sin ninguna conexión activa. En este modo, debemos asignar, a cada "pata" o pin del procesador una función concreta, planificando cómo lo soldaremos a una placa de prototipado a medida.

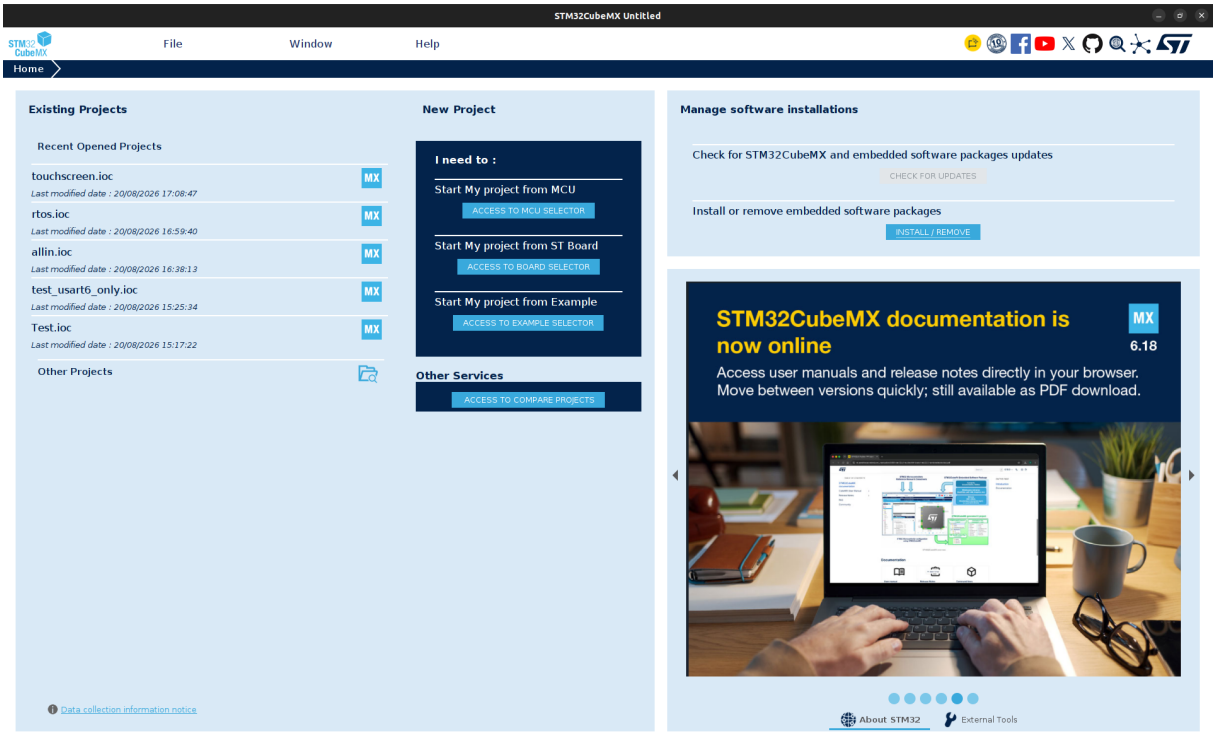


Figura 1: Pantalla inicial de CubeMX

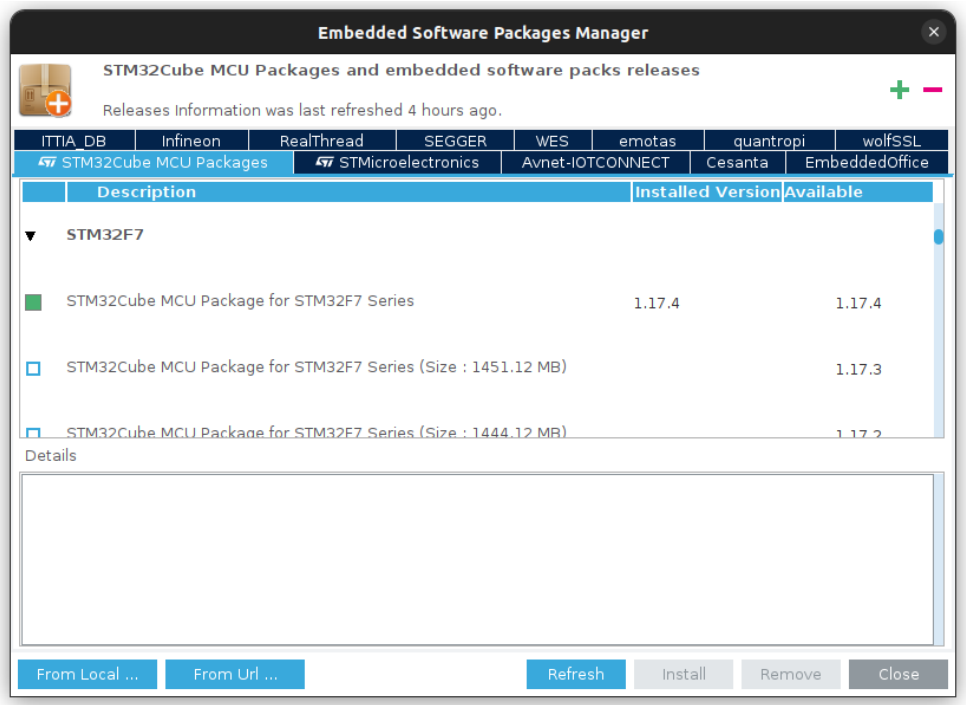


Figura 2: Instalación del paquete STM32F7

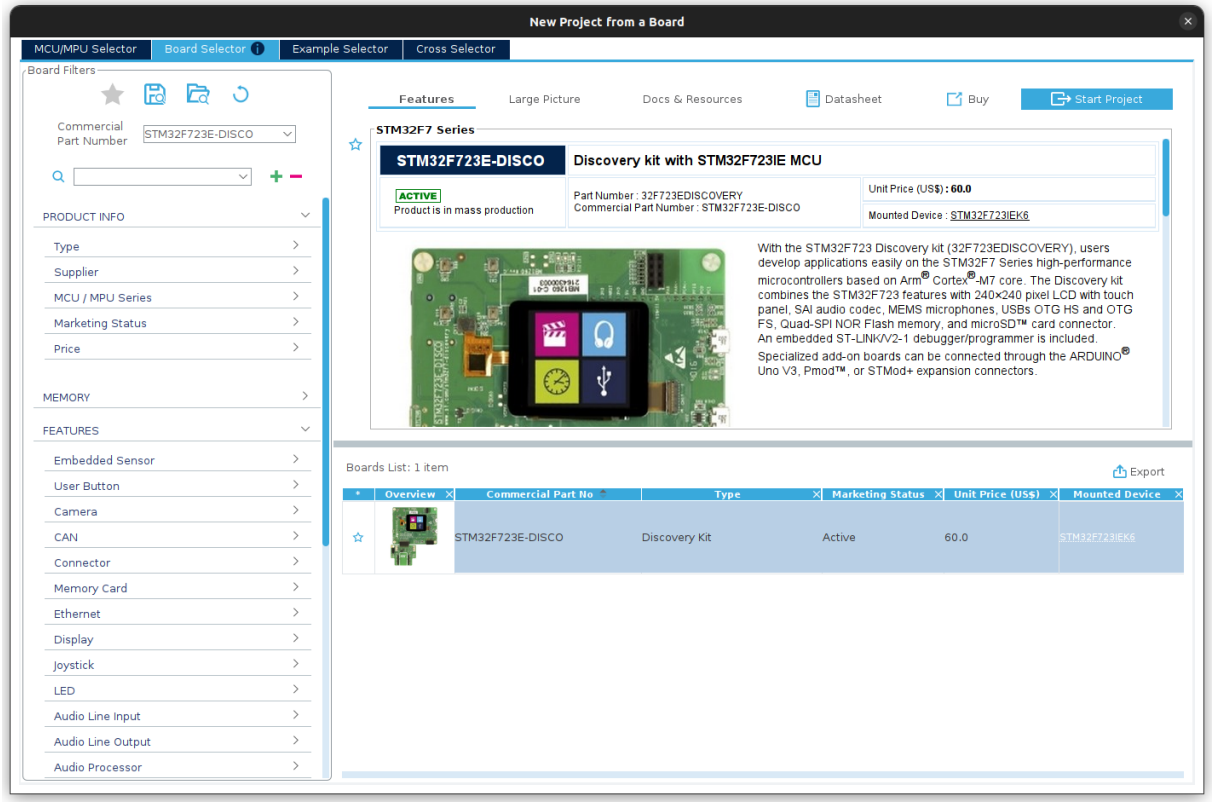


Figura 3: Selector de placas con la STM32F723E-DISCO seleccionada

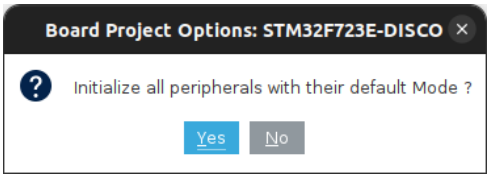


Figura 4: Diálogo de inicialización: seleccionar NO

- **BOARD**: En este caso, partimos de un procesador que ya se encuentra dentro de una placa, y sus pines tienen conexiones concretas establecidas (LEDs, Botones, Pantallas...). En este modo, simplemente deberemos elegir qué funciones activamos, pero no tendremos la dificultad de tener que planificar qué hace cada pin. Utilizaremos este segundo modo eligiendo nuestra placa (723E discovery) (Figura 3).

Al crear el proyecto, nos pedirá una serie de opciones. En concreto, si queremos iniciar **TODOS** los periféricos (Figura 4, diremos que no), y si queremos preconfigurar la MPU (Figura 5, diremos que sí).

Si quisiéramos habilitar TODA la funcionalidad de la placa, deberíamos decir que sí al primer diálogo, pero esto nos hará tener un paquete demasiado grande para las pruebas que queremos hacer.

Una vez creado el proyecto, podemos ver la planificación inicial en la Figura 6:

- Los pines color crema corresponden a alimentación (diferentes niveles de voltaje y tierra).
- Los pines color gris están deshabilitados (no conectados a ningún lado en la placa, por

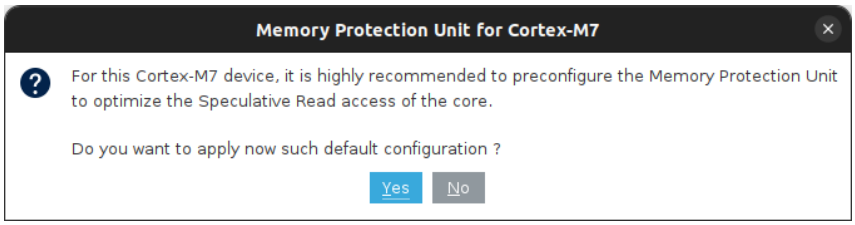


Figura 5: Diálogo de configuración de memoria: seleccionar SÍ

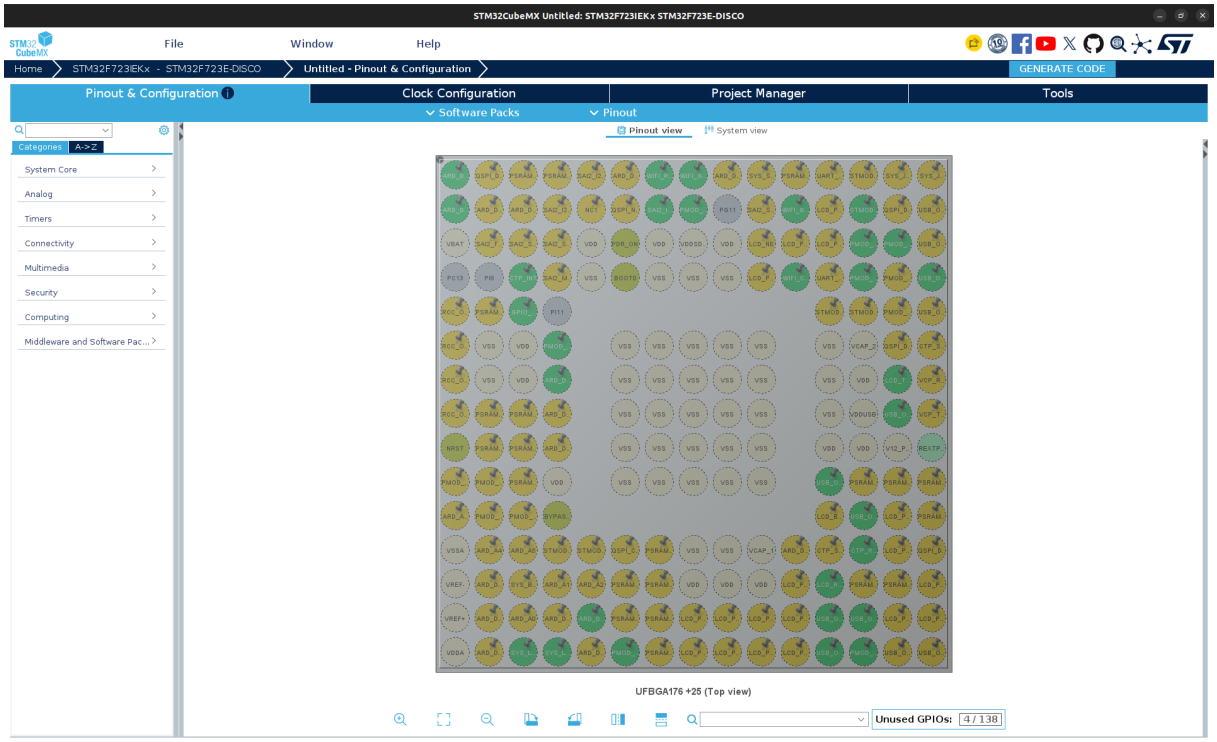


Figura 6: Disposición inicial del procesador en cubemx. Se observan todos los pines y sus funciones

tanto no tienen uso).

- Los pines color verde caqui y aguamarina (solo solo unos pocos) corresponden a funciones de reseteo y la secuencia de arranque.
- Los pines amarillos corresponden a funciones desactivadas pero disponibles (se pueden ir activando como veremos a continuación).
- Los pines verdes corresponden a funciones ya activas y que utilizará nuestra aplicación.

Hay que tener en cuenta que esta configuración de pines es la que se generará automáticamente por la herramienta, lo cual no quita que nosotros podamos cambiarla "al vuelo" dentro de nuestra aplicación, mediante acceso a todos los registros correspondientes de `GPIO`.

Para esta práctica vamos a hacer un par de cambios. El primero es cambiar la función del pin `PA0` para que dispare una interrupción. Deberemos pinchar en él (es el de la posición (3,3) empezando por la esquina inferior izquierda), y seleccionar `GPIO_EXTI0` para habilitar su función de interrupción externa (Figura 7). Finalmente tendremos también que seleccionar la generación de interrupción desde `NVIC` en el menú correspondiente (Figura 8).

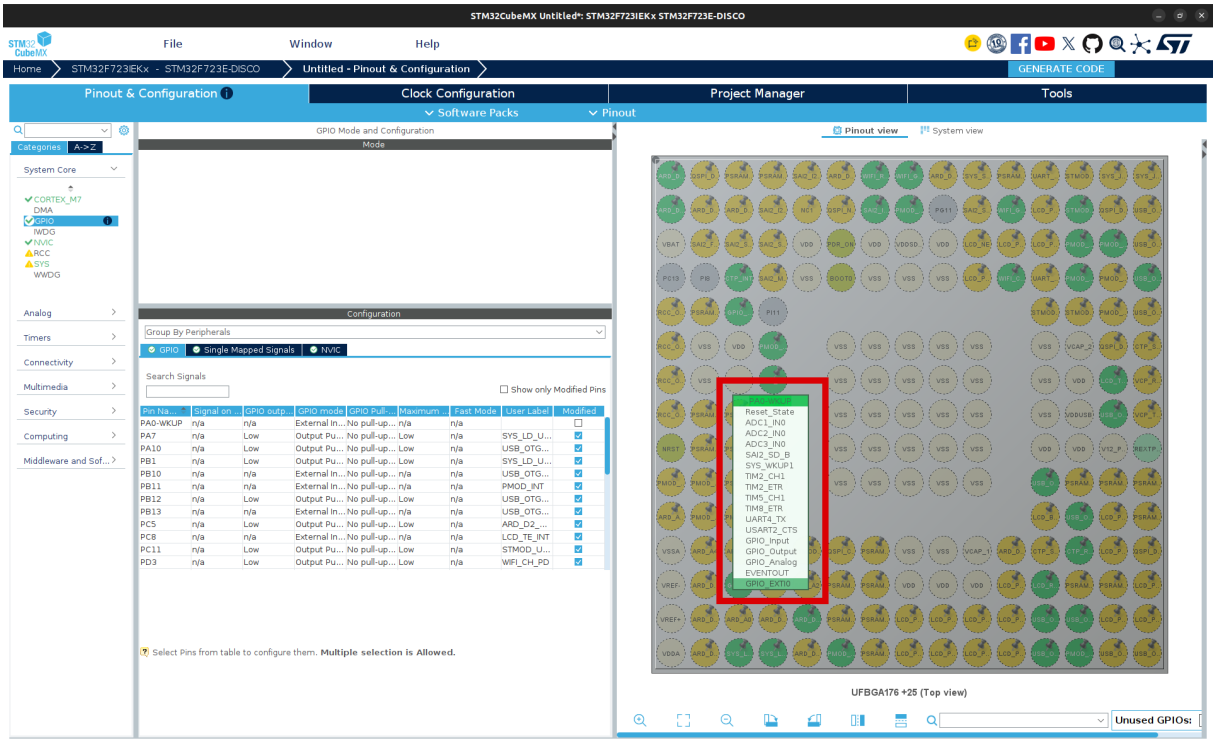


Figura 7: Cambio de función de un pin

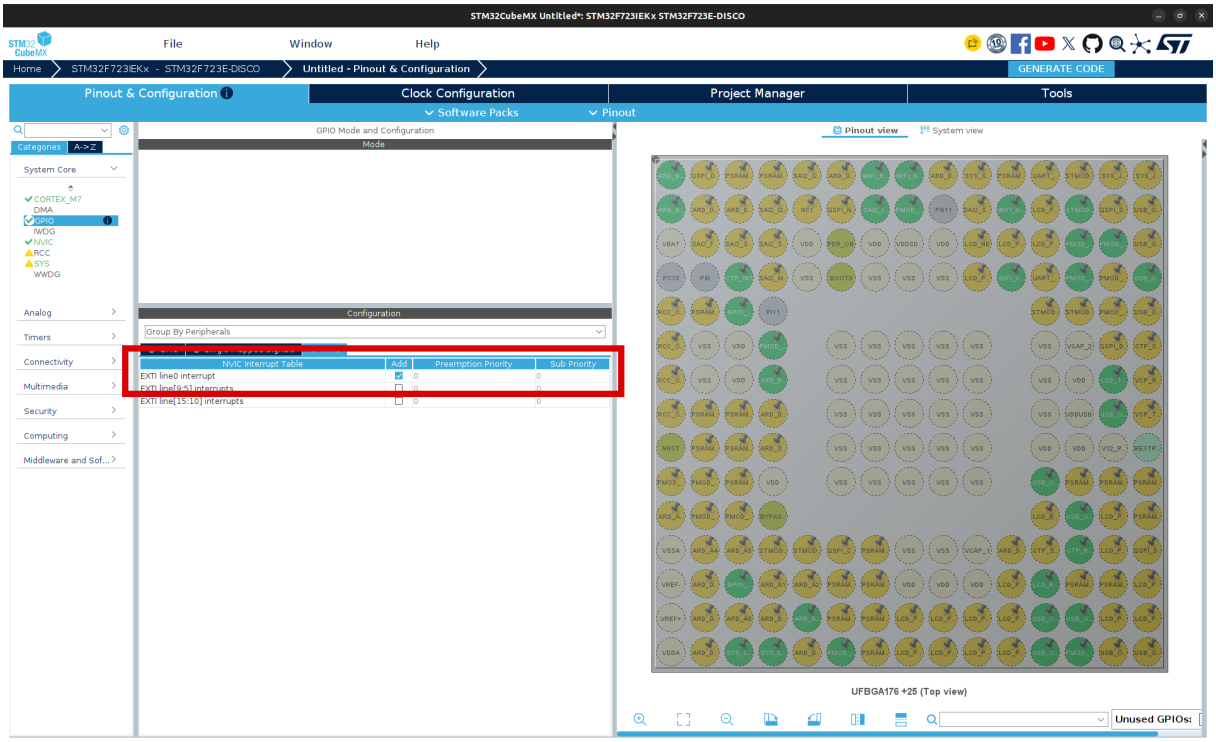


Figura 8: Activación de interrupciones para el pin de `GPIO` A0

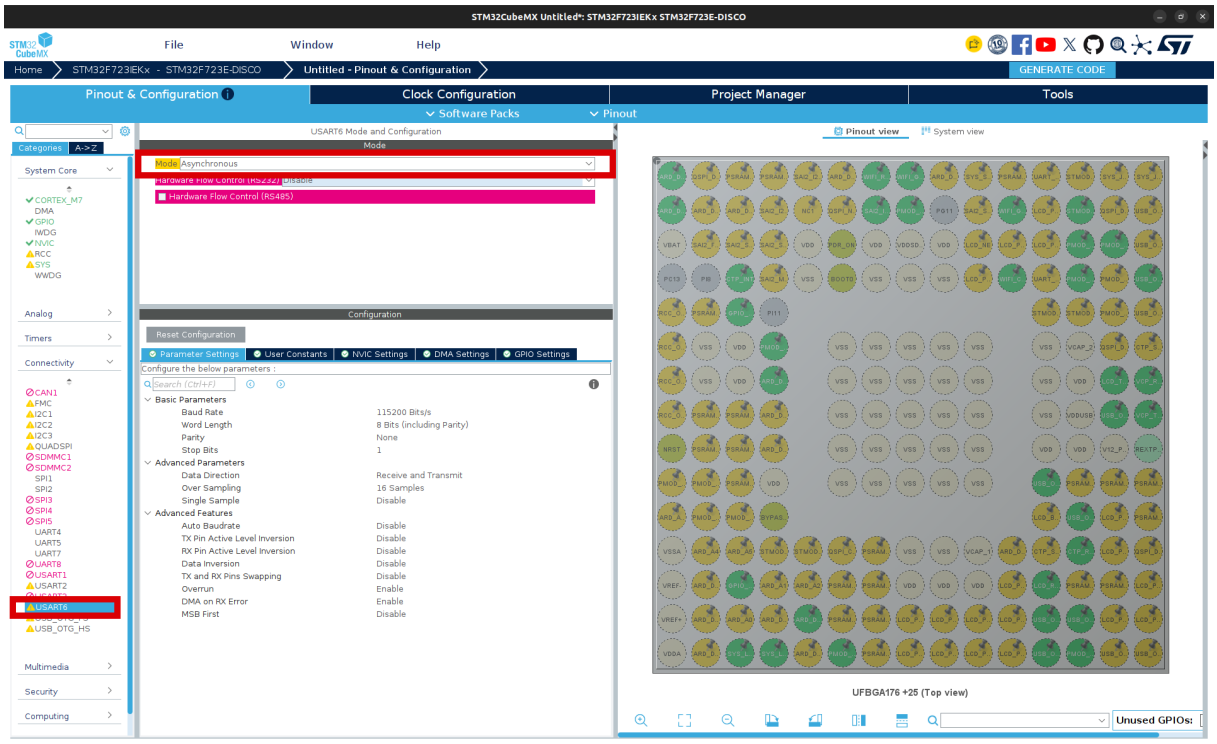


Figura 9: Activación del periférico `USART6`

Además de tratar con los pines, podemos realizar la configuración de todos los periféricos que contiene el procesador. En la barra lateral izquierda, los veremos categorizados. Vamos a activar la `UART6` para poder enviar mensajes al ordenador a través de la conexión serie. Para ello, tenemos que seleccionar en la lista `Connectivity >USART6` y luego activarla poniéndola en modo `Asyncrhonous` (Figura 9).

Veremos que numerosos periféricos y opciones aparecen en color rojo/rosa. Esto es porque la herramienta detecta configuraciones incompatibles que no nos deja cambiar. (Imaginemos por ejemplo que la UART y el bus I2C comparten pines, sólo podríamos tener habilitado uno). Si queremos ver por qué una opción está deshabilitada, basta con dejar el ratón encima, y nos dirá que otras funcionalidades entran en conflicto.

Otra parte bastante potente y que podemos configurar aquí es la gestión del árbol de relojes (Figura 10). El árbol de relojes es la estructura del procesador que toma la oscilación inicial de un cristal o bucle de inversores, y genera frecuencias concretas para todos los elementos internos del procesador. En otras prácticas no hemos estado modificando su configuración, y el reloj por defecto era de 16MHz. Aquí podemos ver que, utilizando diversos multiplicadores y divisores de frecuencia, se consigue un reloj objetivo de 216MHz. Su configuración se generará automáticamente para la placa. Podemos probar a cambiar funciones para ver cómo afecta a todos los relojes del árbol.

Hay que tener en cuenta que, para la configuración de periféricos con reloj propio (como la UART), debemos saber con exactitud la frecuencia de referencia que reciben (por ejemplo, para ajustar valores de contadores), así que un cambio en este punto puede suponer un cambio en nuestro código.

Finalmente, estamos listos para generar el proyecto con nuestro BSP customizado, que incluirá la configuración ya hecha de los periféricos seleccionados y del árbol de reloj. Debemos

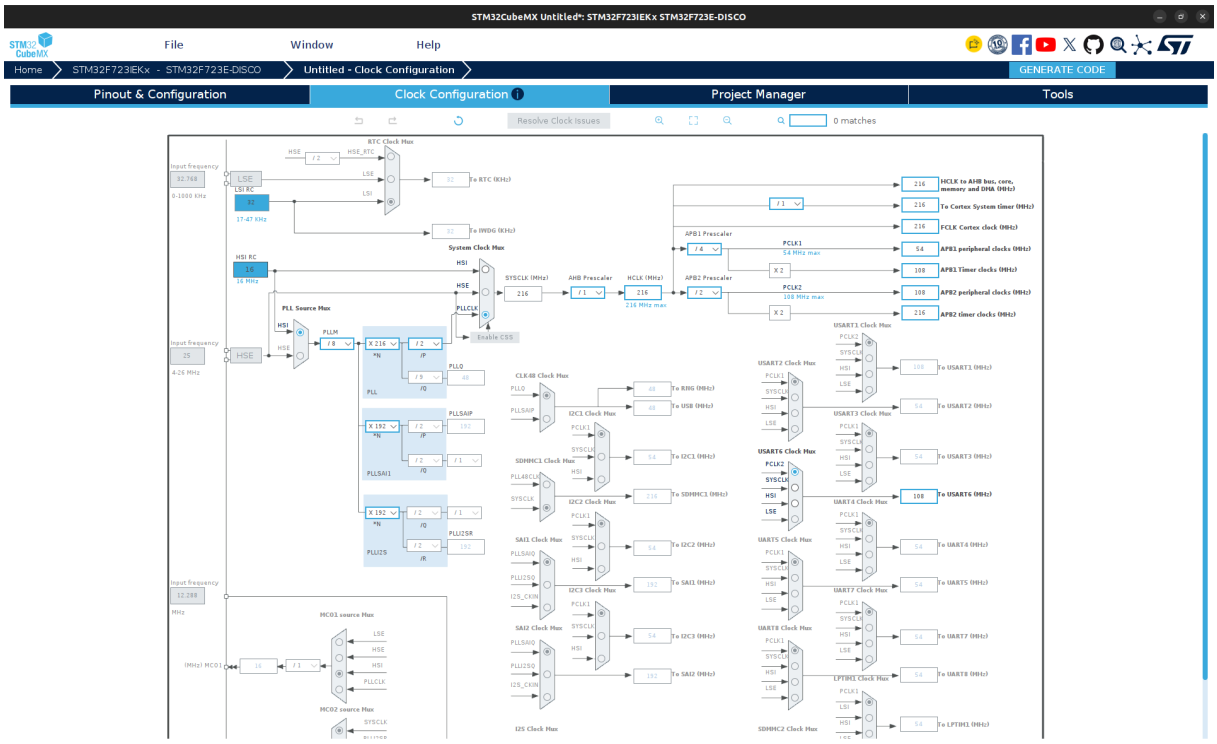


Figura 10: Configuración del reloj

dar un nombre y carpeta al proyecto, además de indicar que lo abriremos con STMCubeIDE (Figura 11). Una vez generado, nos dará la opción directamente de abrir el proyecto (Figura 12).

1.3.2. Uso del BSP desde STMCubeIDE

Ahora ya volvemos al familiar STMCubeIDE. Deberíamos haber visto un diálogo de importación correcta como el de la Figura 13.

En la Figura 14 vemos la estructura de ficheros, así como el código automáticamente generado para la función `main`, que inicializa la MPU, reloj, y periféricos activados (En nuestro caso, algunas inicializaciones genéricas, además de los `GPIO` y `USART6`).

En la Figura 15 podemos ver el código de inicialización del reloj. Una cosa interesante a observar en este BSP, es que suele tener estructuras de configuración complejas (`RCC_OscInitStruct`) que luego ya se vuelcan en registros internos (en este caso del `RCC`) de golpe en las funciones internas.

Para las interrupciones, por ejemplo, vemos que configura el `NVIC` en la Figura 16, y genera el manejador de interrupción en Figura 17. En este caso, por darnos mayor abstracción, ya gestiona automáticamente el borrado de la interrupción pendiente, y nos ofrece una función `__weak` que podemos sobreescribir en nuestro fichero `main.c` para capturar la interrupción.

También vemos cómo ha realizado la configuración de los pines del LED (ya configurado por defecto) (Figura 18) y del botón (Figura 19). También configura multitud de pines extra (para todos los periféricos) aunque no les esté dando uso, ya que no los desactivamos en la vista de *pinout*.

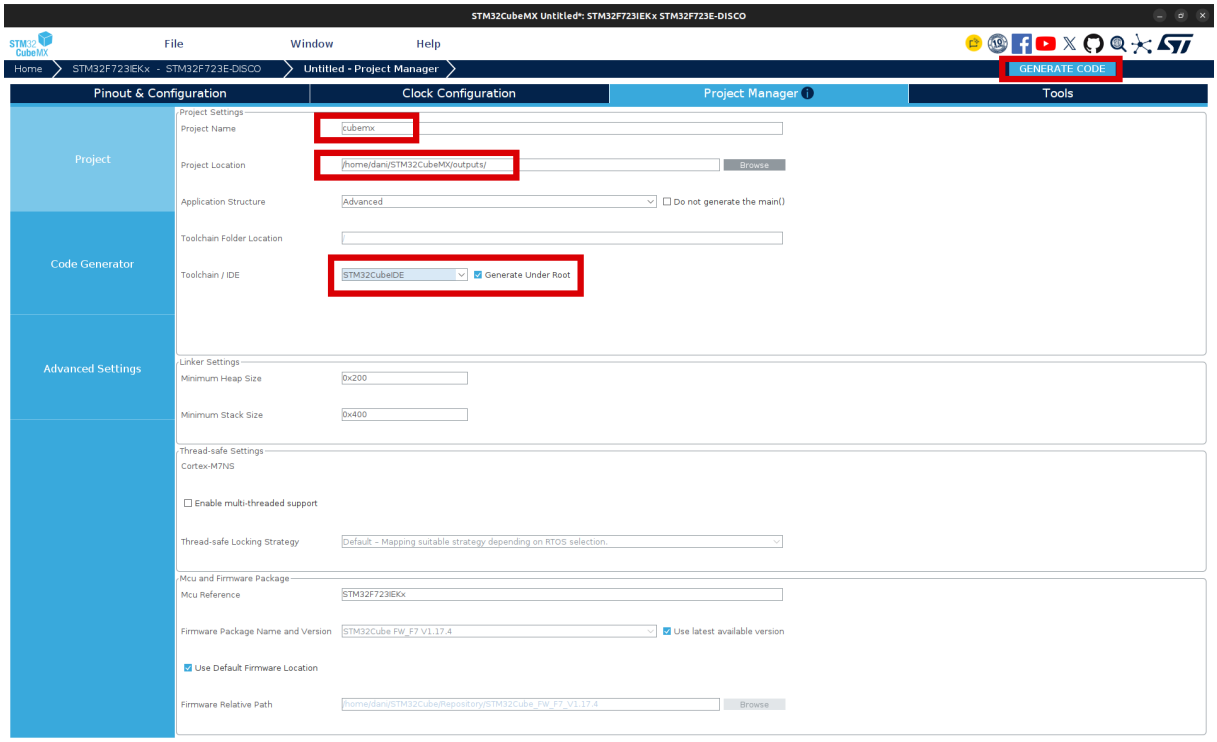


Figura 11: Creación del proyecto con BSP para STMCubeIDE

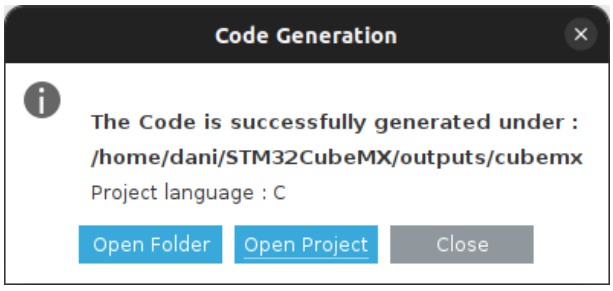


Figura 12: Diálogo de finalización. Seleccionar “Open Project”

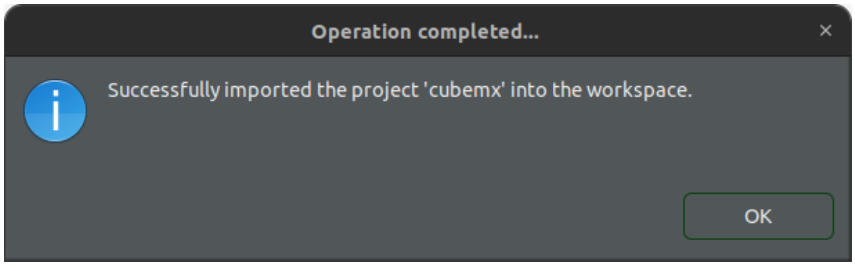


Figura 13: Diálogo de importación exitosa en STMCubeIDE

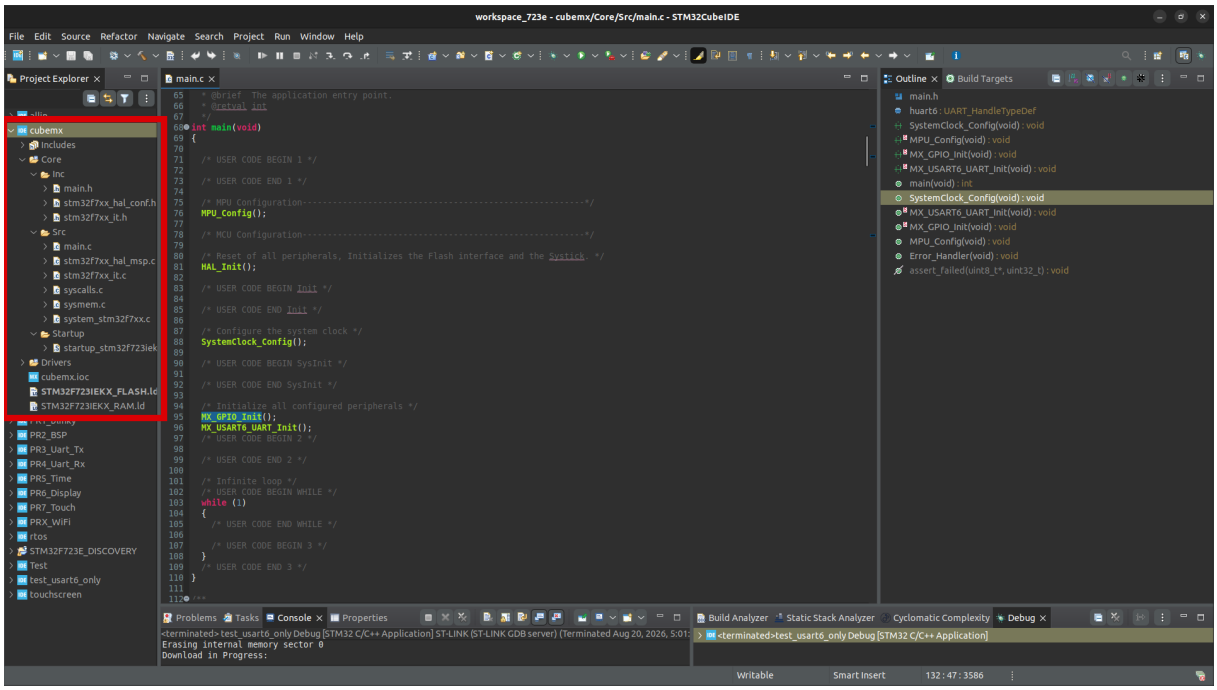


Figura 14: Estructura del proyecto creado y código main

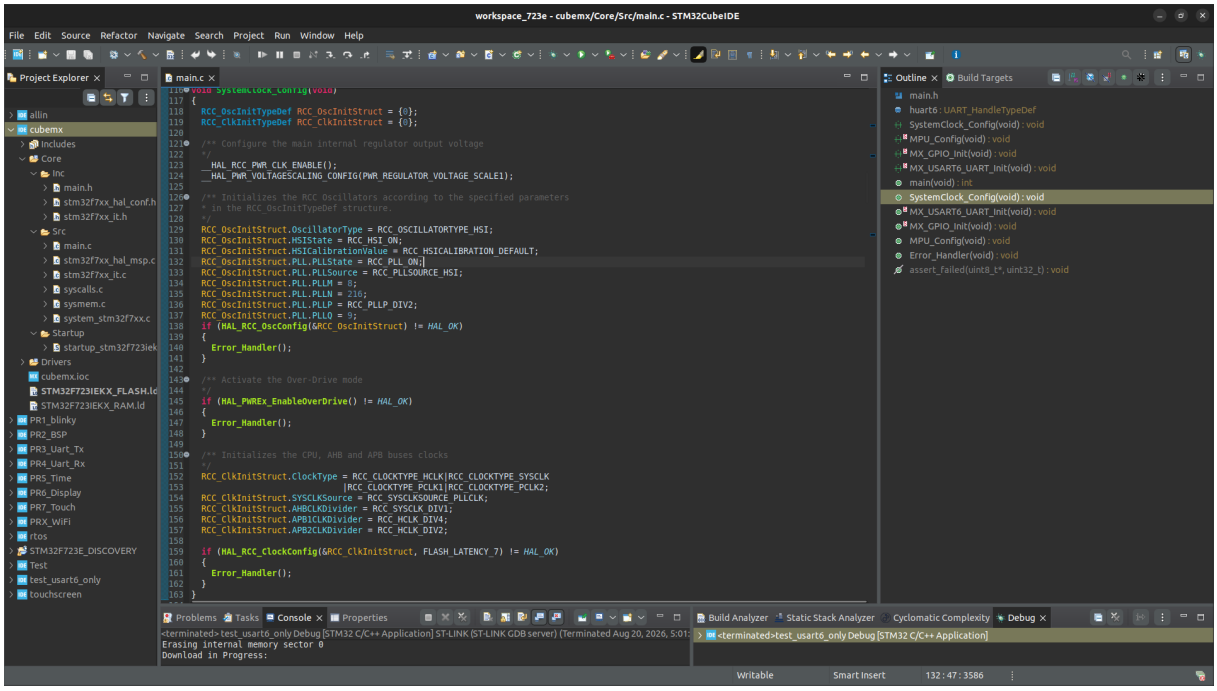


Figura 15: Código creado para inicializar el reloj

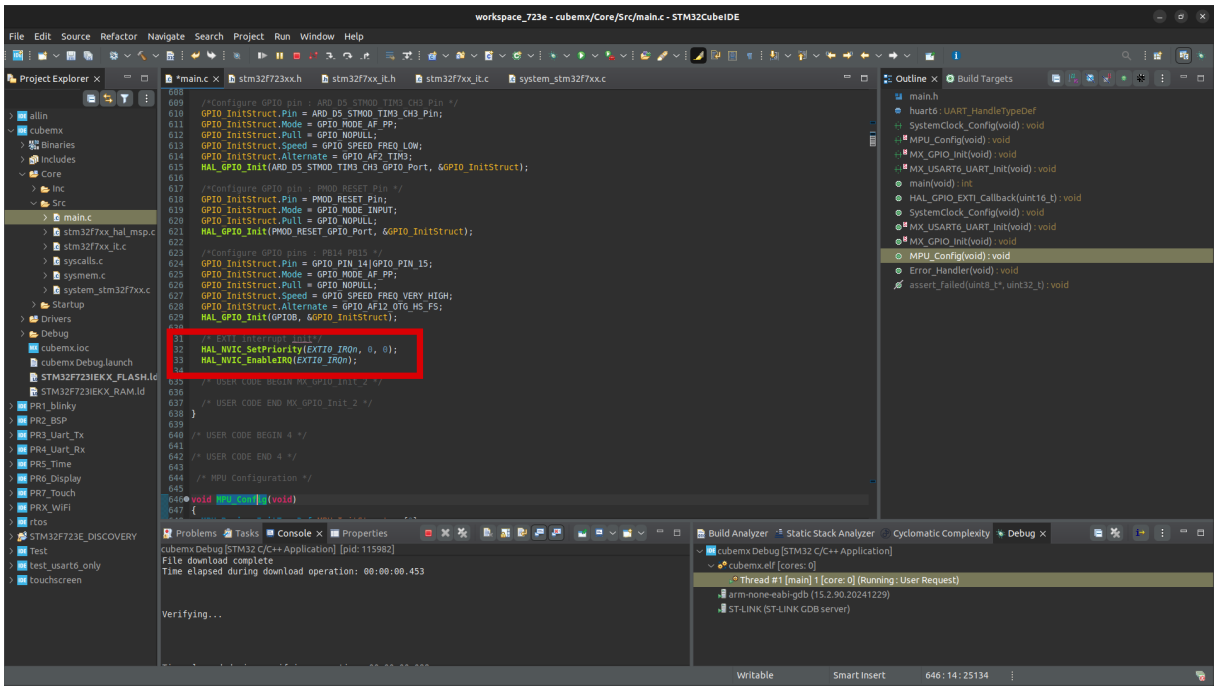


Figura 16: Código generado para inicializar las interrupciones

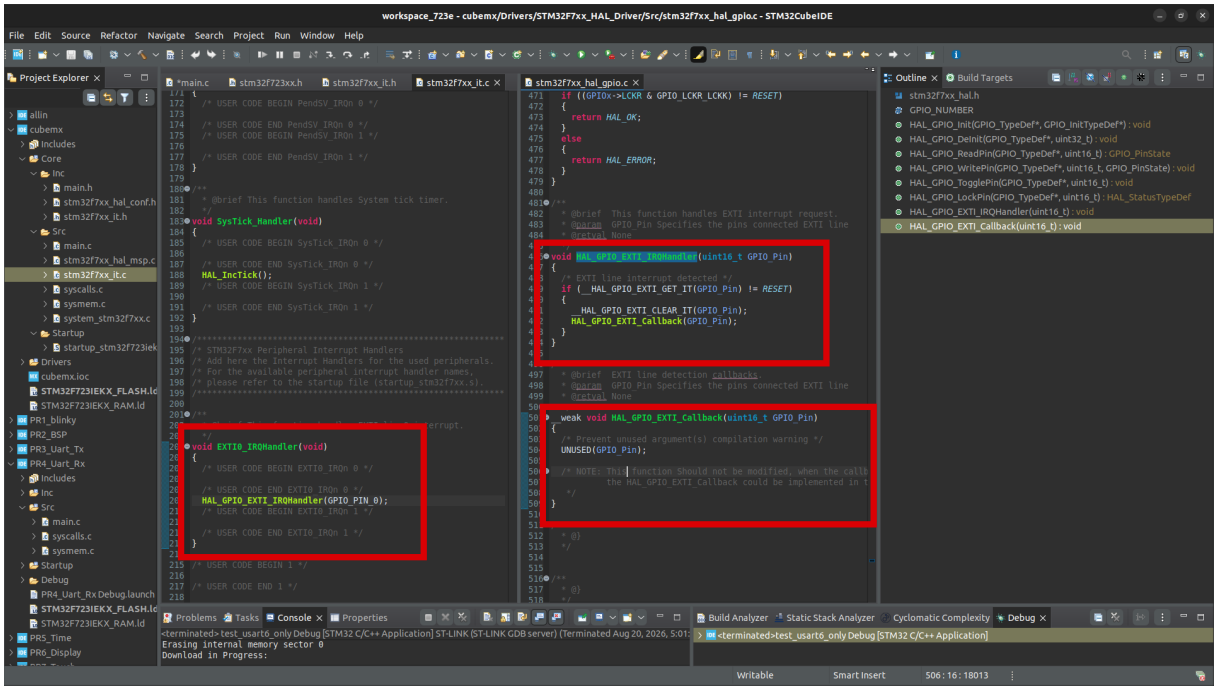


Figura 17: Estructura de las interrupciones, se ofrece mediante un *callback* al usuario

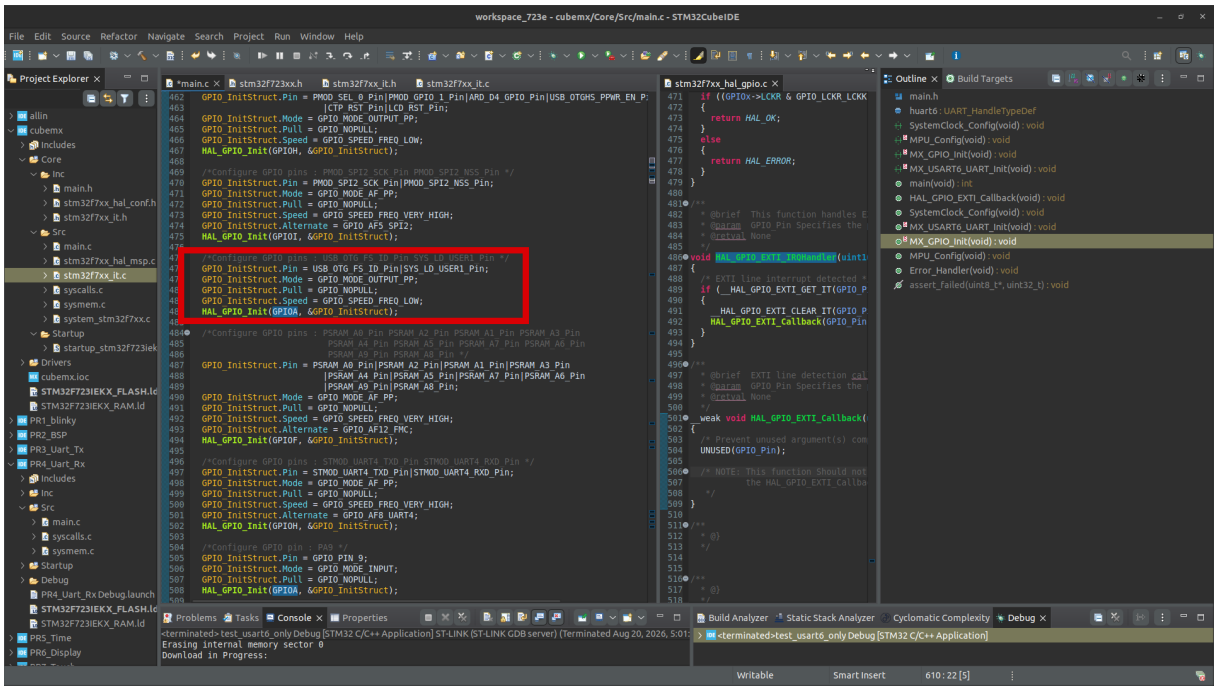


Figura 18: Configuración automática del pin para el LED

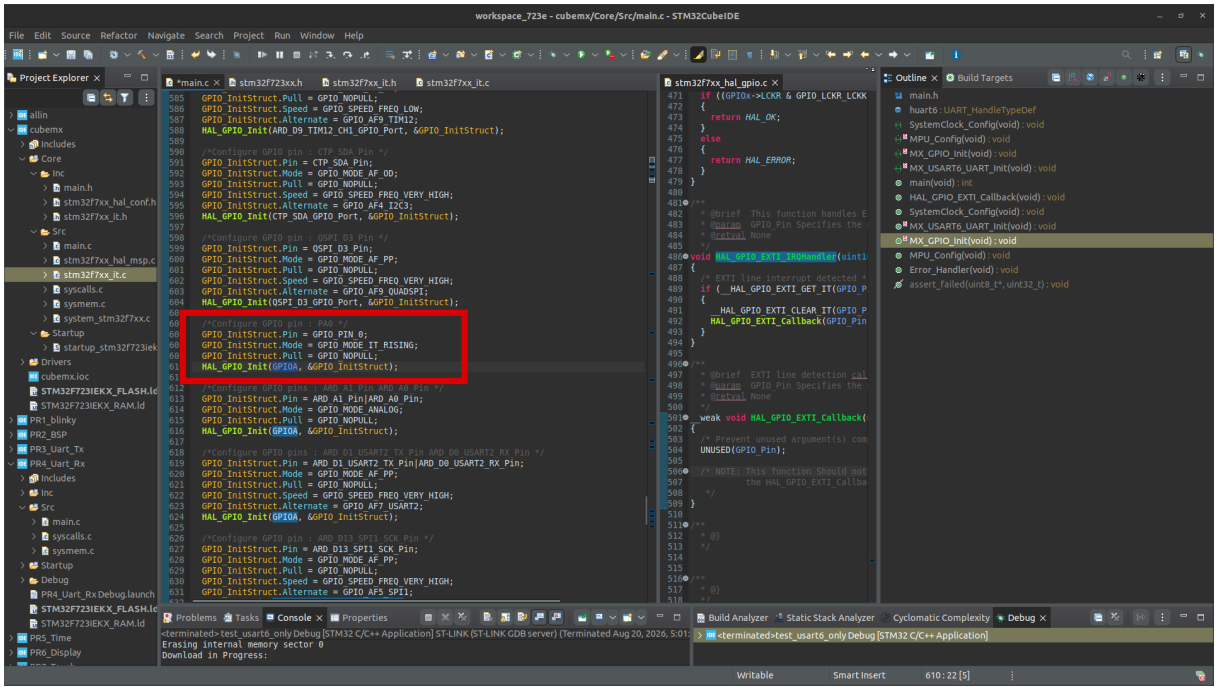


Figura 19: Configuración automática del pin para el Botón

Finalmente, nos queda únicamente hacer uso del paquete que se nos ha dado. Haremos un código sencillo donde, cada cierto tiempo, parpadee un LED. Además, al pulsar el botón, enviaremos por la `UART` un texto.

Si antes esto hubiera supuesto ver la documentación de diferentes periféricos, definir sus estructuras, crear sus funciones auxiliares... ahora podemos directamente hacer uso de las funciones predefinidas en el BSP generado, y nos queda un código muy refinado y corto.

```
int main(void)
{
    /* MPU Configuration--------------------------------------------------------
       */
    MPU_Config();

    /* MCU Configuration--------------------------------------------------------
       */
    /* Reset of all peripherals, Initializes the Flash interface and the Systick
       . */
    HAL_Init();

    /* Configure the system clock */
    SystemClock_Config();

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART6_UART_Init();
    /* Infinite loop */
    while (1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_7);
        HAL_Delay(1000);
    }
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0)
        HAL_UART_Transmit(&huart6, (uint8_t *) "Probando\r\n", 10, 1000);
}
```

1.4. Para hacer más

Investiga cómo podrías añadirle aún más funcionalidad al proyecto incluyendo los ficheros de BSP que están en el repositorio `https://github.com/STMicroelectronics/STM32Cubef7` (`Drivers/BSP` y `Utilities`). Aquí se encuentran drivers de alto nivel para la pantalla o panel táctil, entre otros, así como funciones auxiliares varias.

NOTA: La adición de estos paquetes es manual, y no se incluyen automáticamente desde STMCubeMX. Tendrás que copiar las carpetas que vayas a utilizar a tu proyecto, y añadir sus rutas a la compilación (`botón derecho sobre tu proyecto >properties >C/C++ General >Paths and Symbols`)